

# 软件概要设计说明书

| 项目     | 内容                        |
|--------|---------------------------|
| 文档标识   | SDD-LIFE-ASSIST-2026-V1.0 |
| 版本号    | V1.0                      |
| 日期     | 2026年5月9日                 |
| 适用系统   | 综合生活助手平台（CSCI）            |
| 需求基线参考 | SRS-LIFE-ASSIST-2026-V1.0 |

## 目录

|                  |  |
|------------------|--|
| 软件概要设计说明书        |  |
| 1 引言             |  |
| 1.1 编写目的         |  |
| 1.2 背景           |  |
| 1.3 定义           |  |
| 1.4参考设计          |  |
| 2 系统需求概述         |  |
| 2.1需求规定          |  |
| 2.2业务目标          |  |
| 2.3运行环境          |  |
| 2.3.1设备          |  |
| 2.3.2支持软件        |  |
| 2.4设计约束          |  |
| 2.5功能需求          |  |
| 2.6非功能需求         |  |
| 2.7系统总体架构        |  |
| 3 系统接口设计         |  |
| 3.1 接口概述         |  |
| 3.2接口设计          |  |
| 3.2.1用户接口        |  |
| 3.2.2 与其他软件、硬件接口 |  |
| 3.2.3 接口示例（报文样例） |  |
| 3.3 系统数据结构设计     |  |

|                |       |
|----------------|-------|
| 3.3.1 逻辑结构设计要点 |       |
| 3.3.2 物理结构设计要点 | ..... |
| 3.3.3数据结构与程序关系 | ..... |
| 4 系统数据库设计      | ..... |
| 5 运行设计         | ..... |
| 5.1运行模块组合      | ..... |
| 5.2运行控制        | ..... |
| 5.3运行时间        | ..... |

# 1 引言

## 1.1 编写目的

本文档为《综合生活助手平台》软件概要设计说明书（SDD），在《软件需求规格说明书》（SRS）的基础上给出系统总体设计方案，说明系统的总体架构、模块划分与职责分配、关键业务处理流程、接口与数据结构设计、数据库设计、运行与安全/异常处理设计，为详细设计与后续实现、测试提供依据。

预期读者包括：项目开发成员、测试人员、课程指导教师/评审人员，以及后续维护人员。

## 1.2 背景

a. 待开发软件系统名称：综合生活助手平台（本平台/本系统）。

b. 相关角色与运行现场：

- 任务提出者：课程大作业选题（生活助手平台方向）与课程指导教师
- 开发者：项目小组（孙伟宸、黄羿、孙铭、钟其宏、戴昊）
- 用户：城市居民（普通用户）、本地商家（商家管理员）
- 运行的计算站/中心：本地开发环境与云端测试服务器（Linux），客户端通过桌面/移动浏览器访问 Web/H5

## 1.3 定义

- CSCI：Computer Software Configuration Item，软件配置项（本系统后端服务作为主要 CSCI）
- SRS：Software Requirements Specification，软件需求规格说明书
- SDD：Software Design Description，软件概要设计说明书
- Web/H5：基于浏览器的网页/移动端H5页面

- SPA: Single Page Application, 单页应用
- RESTful API: 一种以资源为中心的HTTP接口风格
- JWT: JSON Web Token, 用于无状态身份鉴权的令牌
- BCrypt: 口令哈希算法, 用于安全存储用户密码
- MySQL: 关系型数据库
- Redis: 内存键值数据库, 用作缓存与临时数据存储
- 订单状态机: 订单从创建到完成/取消的状态流转规则

## 1.4参考设计

- 《综合生活助手平台软件需求规格说明书》（SRS-LIFE-ASSIST-2026-V1.0）
- 《软件开发计划书》（SDP-LIFE-ASSIST-2026-V1.0）
- GB/T 8567-2006 《计算机软件文档编制规范》
- IEEE Std 830-1998 《软件需求规格说明推荐实践》
- RESTful API设计规范（OpenAPI 3.0 思路参考）

# 2 系统需求概述

---

## 2.1需求规定

本系统为面向城市居民与本地商家的本地生活服务平台，提供外卖点餐、到店团购、商家浏览/搜索/推荐、订单与支付（模拟）、评价、商家后台管理等能力。

主要输入/输出与处理要求如下：

- 主要输入
  - 普通用户：注册/登录信息、搜索关键词、定位信息（经纬度）、购物车操作、下单信息、支付确认、评价内容、收货地址
  - 商家管理员：商家信息维护、菜品/套餐维护、订单处理（接单/完成配送/核销券码）
- 主要输出
  - 普通用户：商家列表与详情、菜品/套餐列表、购物车与订单信息、订单状态、支付结果、评价列表、个人信息
  - 商家管理员：订单列表与处理结果、商家/菜品/套餐维护结果
- 关键性能/容量（测试环境目标）
  - 并发：≥50并发用户
  - 响应时间：普通查询类接口≤1s；搜索类接口≤2s；首屏加载≤3s
  - 数据量：用户≤1000、商家≤100、菜品≤5000、订单≤10000（用于课程测试/演示）

## 2.2业务目标

- 为用户提供“吃、喝、玩、乐、购”一体化入口，降低信息分散与决策成本
- 为商家提供基础数字化经营工具，提高曝光与订单转化
- 以课程项目为范围，优先交付关键闭环：浏览/搜索 → 加购 → 下单 → （模拟）支付 → 状态跟踪 → 完成 → 评价

## 2.3运行环境

系统采用前后端分离架构：客户端通过桌面/移动浏览器访问 Web/H5 前端；前端通过 RESTful API 调用后端服务；后端以 Spring Boot 运行并访问 MySQL 与 Redis。

### 2.3.1设备

- 服务端（云端测试服务器最低配置）
  - 2核CPU、4GB RAM、50GB存储、100Mbps带宽
- 客户端
  - 桌面：支持现代浏览器的PC
  - 移动端：Android 8.0+/iOS 13+，支持系统浏览器访问H5

### 2.3.2支持软件

- 服务端：Linux（CentOS 7.x/Ubuntu 20.04+）、JDK 17、Spring Boot 3.x、MySQL 8.x、Redis 7.x、（可选）Nginx 1.20+
- 前端构建：Node.js 18.x、（前端框架）Vue 3.x 或 React 18.x
- 客户端：Chrome 90+/Firefox 88+/Safari 14+/Edge 90+

## 2.4设计约束

- 架构：前后端分离，后端提供 RESTful API；分层调用（Controller 不直接访问 Mapper）
- 技术栈：后端 Java 17 + Spring Boot 3.x；ORM 使用 MyBatis-Plus；数据库 MySQL 8.x；缓存 Redis 7.x
- 支付：不接入真实支付渠道，采用模拟支付，预留扩展接口
- 安全：密码 BCrypt 存储；JWT 鉴权；日志与展示脱敏；敏感配置使用环境变量/独立配置
- 项目范围：课程实验项目，不进行商业化运营，仅测试环境部署

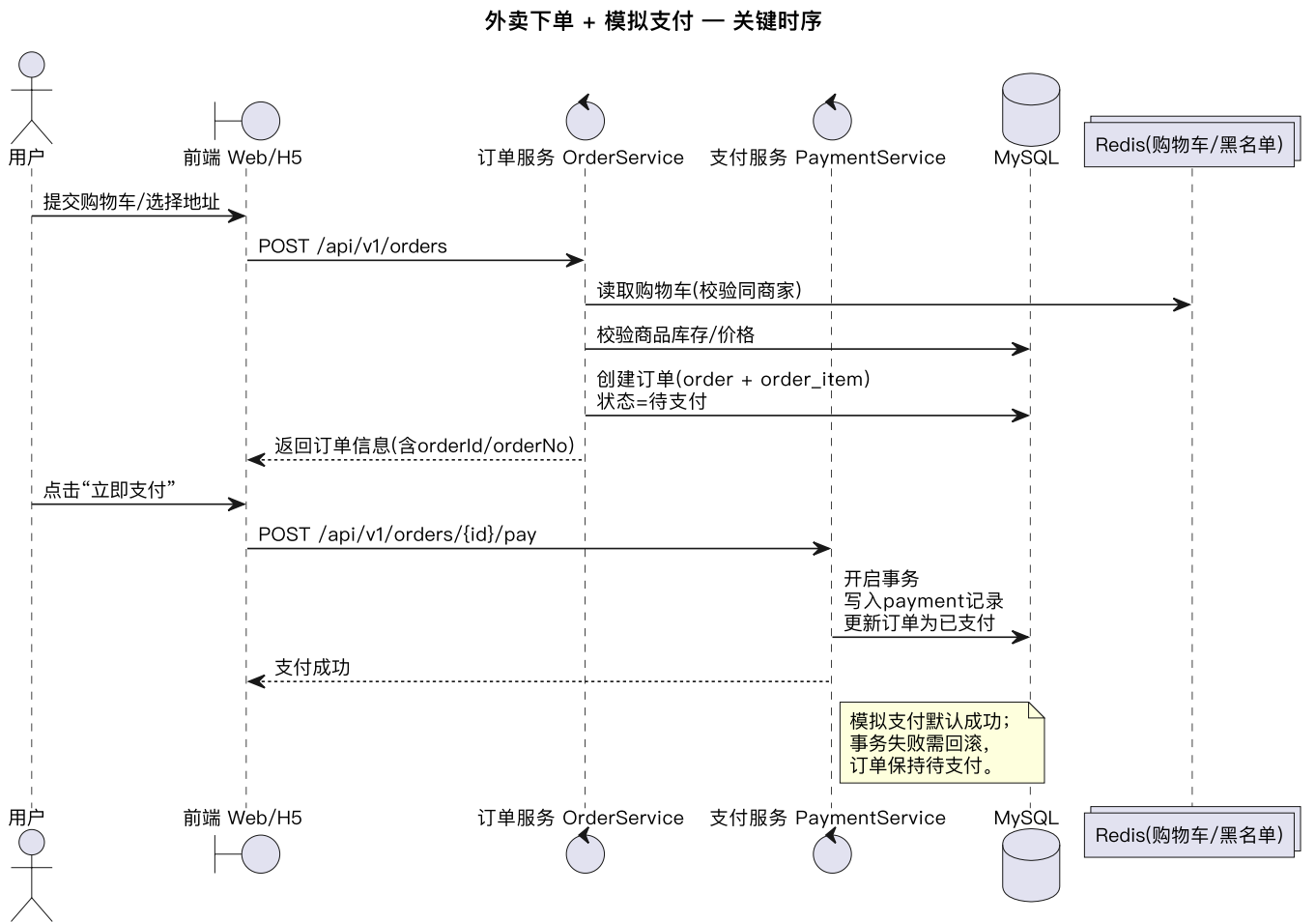
## 2.5功能需求

系统按业务域划分为：用户（User）、商家（Merchant）、订单（Order，含外卖/团购）、支付（Payment，模拟）、评价（Review）、商家后台管理（Management）等模块。核心业务闭环

如下：

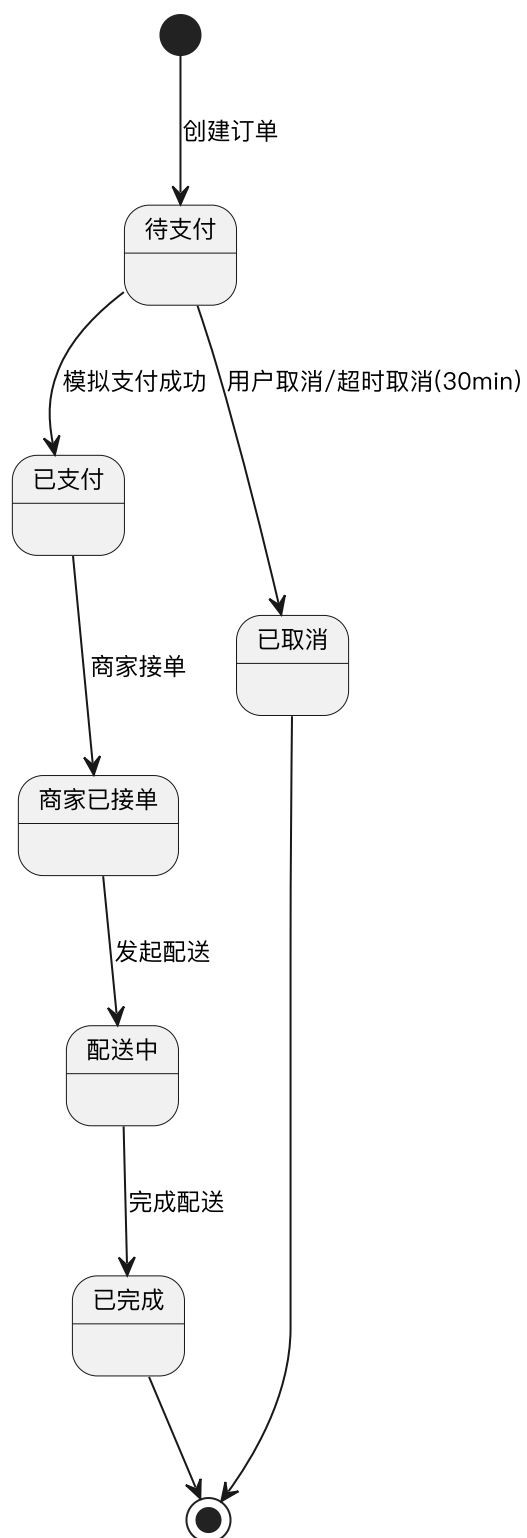
1. 用户注册/登录获取 JWT → 2) 浏览/搜索商家与菜品 → 3) 购物车加购 → 4) 创建订单 → 5) 模拟支付 → 6) 订单状态流转（商家接单/配送/核销） → 7) 完成后用户评价。

外卖下单与支付的关键时序（概要）：



订单状态机（外卖）在概要设计中采用状态流转约束，确保不允许越级或非法状态变更（如已支付订单直接取消）。

### 外卖订单状态机（概要）



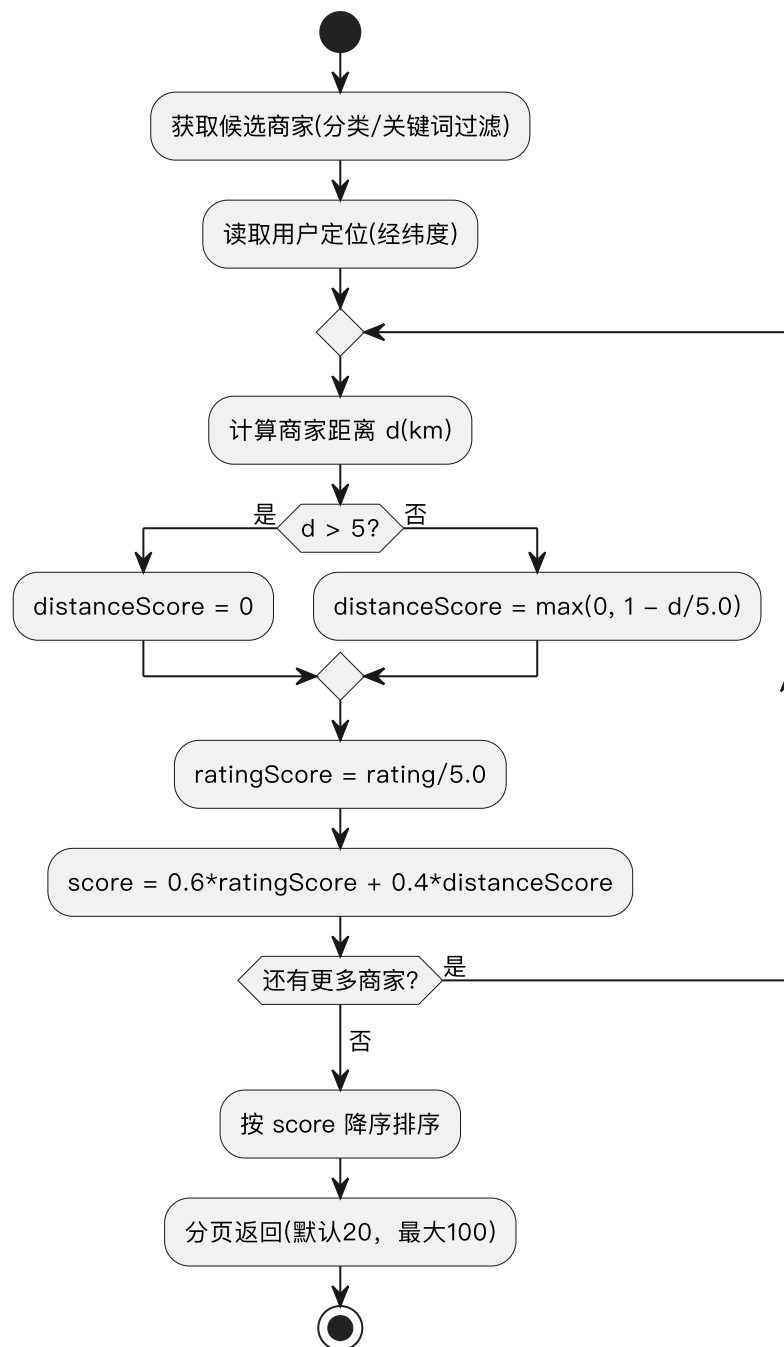
### 商家推荐排序算法（概要）：

- 综合评分计算：

$$Score = 0.6 \times \frac{rating}{5.0} + 0.4 \times \max(0, 1 - \frac{distanceKm}{5.0})$$

- 距离超过 5km 的商家在距离维度得分为 0（降权），以保证“附近优先”。

商家推荐排序算法（概要流程）



## 2.6非功能需求

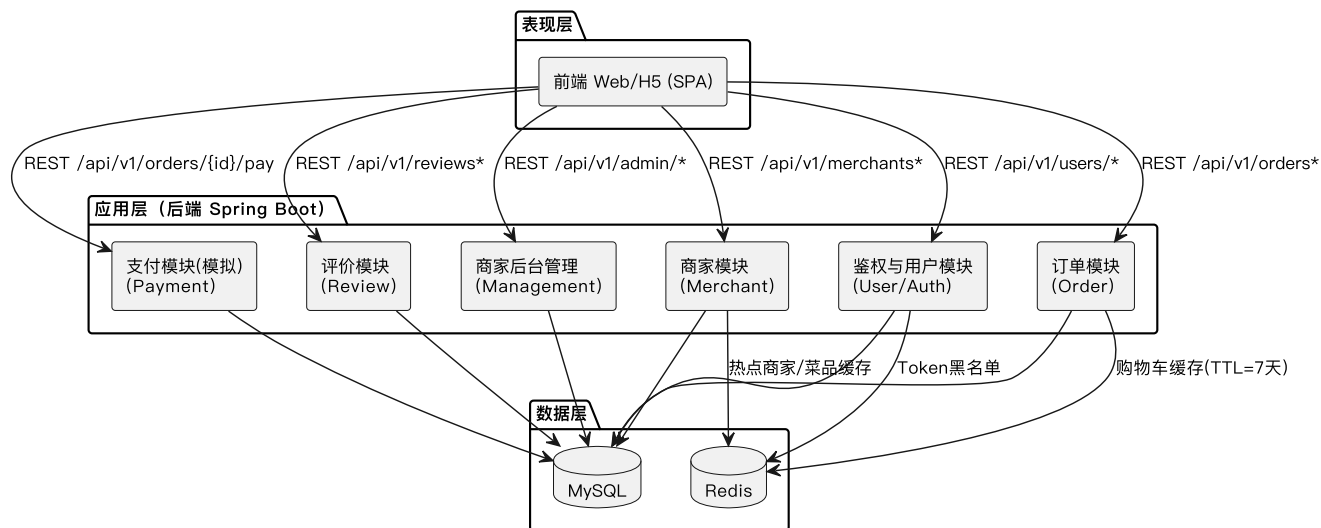
- 性能：查询接口 $\leq 1s$ 、搜索 $\leq 2s$ （50并发）；支持分页（默认20，最大100）
- 可靠性：订单与支付更新需事务一致性；异常需回滚；避免脏数据
- 安全：BCrypt 密码；JWT 鉴权；越权访问拦截；输入校验与XSS防护；支付幂等
- 可维护性：分层架构、统一响应与异常处理、模块间通过接口解耦

- 可扩展性：支付预留真实渠道适配接口；推荐排序封装为策略

## 2.7 系统总体架构

系统采用“表现层（前端 Web/H5）—应用层（Spring Boot 服务）—数据层（MySQL/Redis）”的三层架构，并以模块化方式组织业务能力。

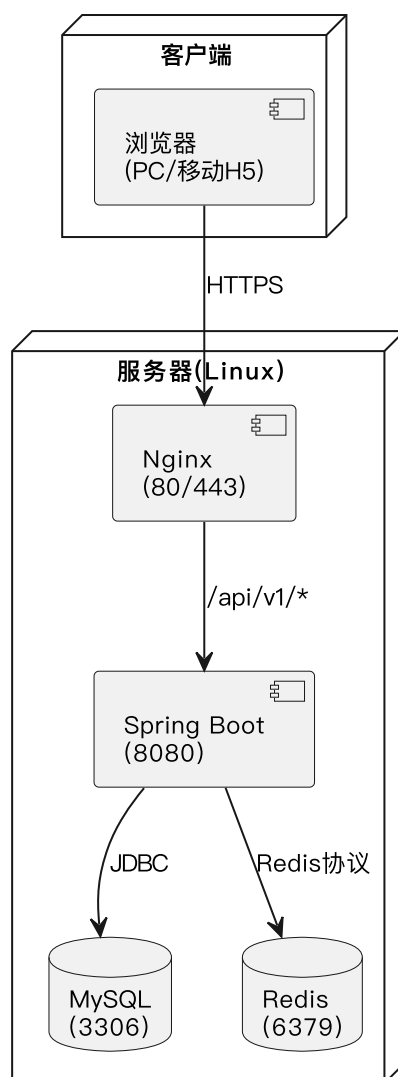
综合生活助手平台 — 总体架构（组件图）



部署（测试环境）建议：Nginx 承载前端静态资源并反向代理 `/api/v1/*` 至后端 8080 端口；后端访问 MySQL 3306、Redis 6379。



综合生活助手平台 — 部署图（概要）



## 3 系统接口设计

### 3.1 接口概述

系统对外主要以 RESTful API 形式提供服务，统一路径前缀为 `/api/v1/`，数据格式为 JSON (UTF-8)，鉴权通过 `Authorization: Bearer <JWT>` 请求头实现。系统内部模块通过 Service 接口调用，禁止跨层直接访问数据层。

### 3.2接口设计

#### 3.2.1用户接口

本系统用户接口为 Web/H5 图形界面，不提供命令行接口。

普通用户端主要页面/交互：

- 注册/登录页：手机号、密码、昵称（注册）输入与校验；登录成功保存 Token
- 首页/分类页：商家分类入口、推荐列表、定位信息展示
- 搜索页：关键词搜索商家/菜品，展示排序结果与空状态提示
- 商家详情页：商家信息、菜品/套餐列表、加入购物车、评价列表
- 购物车/确认订单页：数量调整、跨商家加购提示、选择收货地址、提交订单
- 支付页（模拟）：展示金额，确认后完成模拟支付
- 我的订单：订单列表、订单详情、状态跟踪、待支付取消
- 评价页：对已完成订单提交评分/文字/（可选）图片

商家管理员端主要页面/交互：

- 商家信息维护：修改名称、地址、营业时间、主图等
- 菜品/套餐管理：新增、编辑、上下架、库存维护
- 订单管理：外卖订单接单/完成配送；团购订单核销券码

用户可见的统一响应规则（前端展示）：成功提示/页面刷新数据；失败时展示中文错误信息（不暴露堆栈），并在表单/操作区域附近提示。

### 3.2.2 与其他软件、硬件接口

- 与前端（Web/H5）接口：HTTP/HTTPS + RESTful API, JSON；分页参数 `page / size`
- 与 MySQL 接口：JDBC（MyBatis-Plus），事务保证订单/支付/评价一致性
- 与 Redis 接口：缓存热点数据（商家/菜品）、购物车（TTL=7天）、JWT黑名单（TTL=Token剩余有效期）
- 与模拟支付接口：后端内部模块调用（或独立服务HTTP调用），默认返回支付成功；预留真实支付适配接口

对外API（概要列举）：

- 用户： `POST /api/v1/users/register`、 `POST /api/v1/users/login`
- 商家： `GET /api/v1/merchants`、 `GET /api/v1/merchants/{id}`、 `GET /api/v1/merchants/search`
- 购物车： `GET/POST/DELETE /api/v1/cart`
- 订单： `GET/POST /api/v1/orders`、 `POST /api/v1/orders/{id}/pay`
- 评价： `POST /api/v1/reviews`、 `GET /api/v1/reviews/merchant/{id}`

### 3.2.3 接口示例（报文样例）

为便于前后端联调与后端实现，本节给出典型接口的请求/响应示例。示例仅展示关键字段，字段含义与约束以SRS为准。

## (1) 通用约定

- 请求头（已登录接口）：

```
Authorization: Bearer <JWT_Token>
Content-Type: application/json
Accept: application/json
```

- 分页约定：page 从 1 开始；size 默认 20，最大 100。
- 统一响应格式：

```
{
  "code": 200,
  "message": "success",
  "data": {}
}
```

- 统一错误响应示例：

```
{
  "code": 401,
  "message": "未授权：请先登录",
  "data": null
}
```

常用错误码：400 参数错误、401 未授权、403 无权限、404 资源不存在、409 业务冲突、500 服务器内部错误。

## (2) 用户注册（USR-FUN-001）

- POST /api/v1/users/register

请求：

```
{
  "phone": "13800138000",
  "password": "abc12345",
  "nickname": "小明"
}
```

成功响应：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "userId": 10001
  }
}
```

重复手机号（409）示例：

```
{
  "code": 409,
  "message": "该手机号已注册",
  "data": null
}
```

### (3) 用户登录（USR-FUN-002）

- POST /api/v1/users/login

请求：

```
{
  "phone": "13800138000",
  "password": "abc12345"
}
```

成功响应（返回JWT，有效期24小时）：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "expiresIn": 86400,
    "user": {
      "userId": 10001,
      "nickname": "小明",
      "role": "USER"
    }
  }
}
```

#### (4) 商家列表 (MCH-FUN-001)

- GET /api/v1/merchants?page=1&size=20&lat=31.2304&lng=121.4737&categoryId=10

成功响应：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "page": 1,
    "size": 20,
    "total": 2,
    "items": [
      {
        "merchantId": 20001,
        "name": "张记小馆",
        "categoryId": 10,
        "avgScore": 4.6,
        "distanceKm": 1.2,
        "address": "某某路88号",
        "status": "营业中"
      },
      {
        "merchantId": 20002,
        "name": "奶茶研究所",
        "categoryId": 12,
        "avgScore": 4.8,
        "distanceKm": 3.4,
        "address": "某某街66号",
        "status": "休息中"
      }
    ]
  }
}
```

#### (5) 商家关键词搜索 (MCH-FUN-003)

- GET /api/v1/merchants/search?q=奶茶&page=1&size=20&lat=31.2304&lng=121.4737

响应与商家列表一致（按相关度/综合排序返回）。

#### (6) 购物车加购 (ORD-FUN-001)

- POST /api/v1/cart/items

请求：

```
{
  "merchantId": 20001,
  "productId": 30001,
  "quantity": 2
}
```

成功响应（返回当前购物车摘要）：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "merchantId": 20001,
    "items": [
      {
        "productId": 30001,
        "productName": "宫保鸡丁",
        "price": 2600,
        "quantity": 2
      }
    ],
    "totalAmount": 5200
  }
}
```

跨商家加购冲突（409）示例：

```
{
  "code": 409,
  "message": "购物车已有其他商家商品，是否清空后继续",
  "data": {
    "currentMerchantId": 20001,
    "incomingMerchantId": 20002
  }
}
```

## (7) 创建外卖订单（ORD-FUN-002）

- `POST /api/v1/orders`

请求：

```
{
  "type": "外卖",
  "merchantId": 20001,
  "addressId": 50001,
  "remark": "少辣",
  "items": [
    { "productId": 30001, "quantity": 2 },
    { "productId": 30002, "quantity": 1 }
  ]
}
```

成功响应：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "orderId": 70001,
    "orderNo": "20260509143015987654",
    "status": "待支付",
    "totalAmount": 7800
  }
}
```

商品下架/库存不足（400）示例：

```
{
  "code": 400,
  "message": "以下商品已下架：宫保鸡丁，请重新选择",
  "data": null
}
```

## (8) 发起模拟支付（PAY-FUN-001/002）

- POST /api/v1/orders/{orderId}/pay

请求：



```
{
  "payMethod": "MOCK",
  "clientRequestId": "c9b1f7b0-9c1a-4f1e-8f6c-2d0f0c0a1a11"
}
```

成功响应：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "paymentId": 80001,
    "orderId": 70001,
    "paidAt": "2026-05-09T14:31:02+08:00",
    "status": "成功"
  }
}
```

说明：`clientRequestId` 用于幂等控制（重复点击支付不应产生多条支付记录）。

## (9) 提交评价 (REV-FUN-001)

- `POST /api/v1/reviews`

请求：

```
{
  "orderId": 70001,
  "overallScore": 5,
  "tasteScore": 5,
  "serviceScore": 4,
  "content": "味道不错，配送很快",
  "imageUrls": []
}
```

成功响应：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "reviewId": 90001
  }
}
```

重复评价（409）示例：

```
{
  "code": 409,
  "message": "您已对该订单进行过评价",
  "data": null
}
```

## (10) 商家后台：上下架菜品（MGT-FUN-002）

- `PUT /api/v1/admin/products/{productId}`

请求：

```
{
  "name": "宫保鸡丁",
  "price": 2600,
  "stock": 99,
  "isOnSale": true
}
```

成功响应：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "productId": 30001,
    "isOnSale": true,
    "updatedAt": "2026-05-09T14:40:00+08:00"
  }
}
```

## 3.3 系统数据结构设计

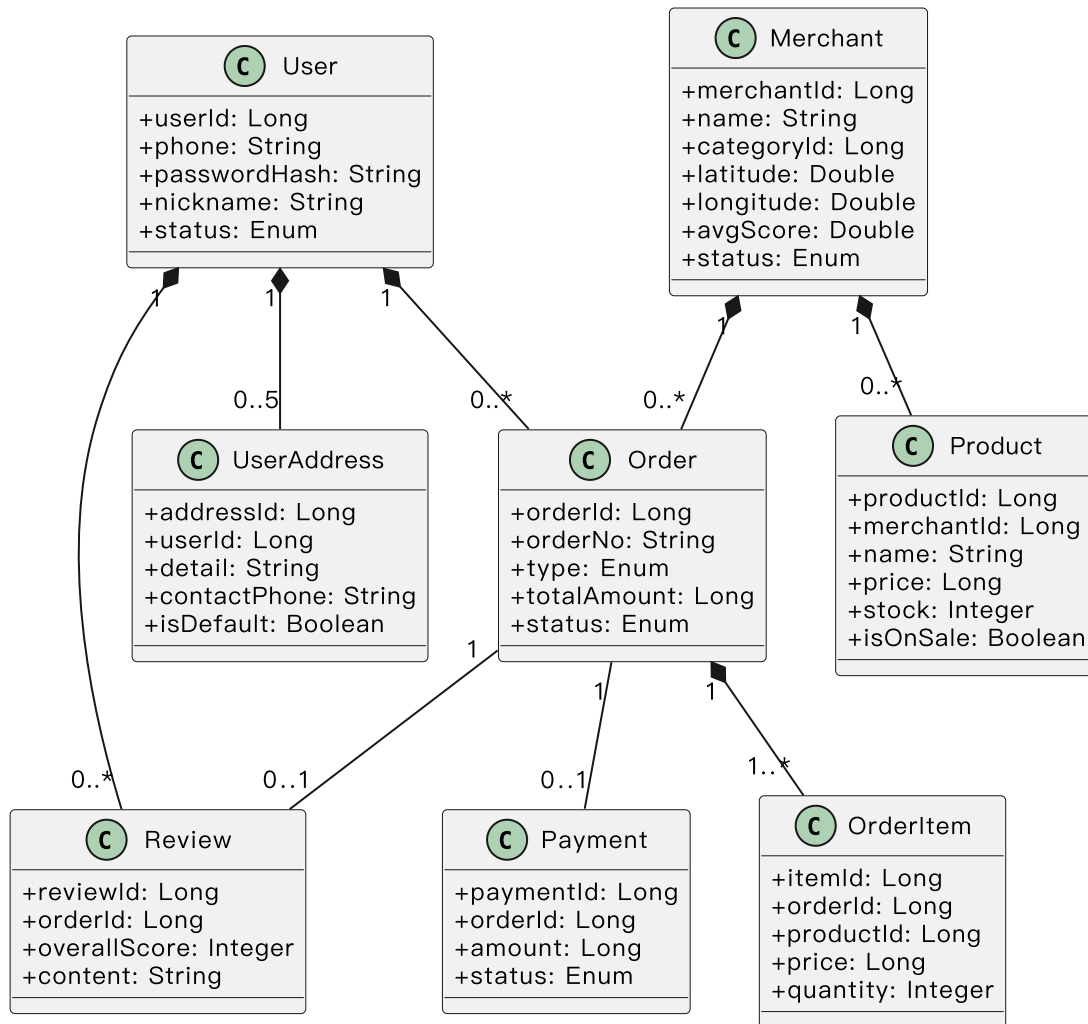
### 3.3.1 逻辑结构设计要点

系统核心业务对象（逻辑模型）如下：

- User：用户（手机号唯一；密码哈希存储；状态正常/锁定）
- UserAddress：收货地址（≤5条；默认地址标记）
- Merchant：商家（分类、经纬度、营业时间、综合评分、状态）
- Category：商家分类
- Product：菜品/套餐（价格以“分”存储；库存；上架状态；销量）
- Cart：购物车（按用户+商家维度；仅支持同商家；存Redis）
- Order：订单（外卖/团购；总金额；状态；地址快照）
- OrderItem：订单明细
- Payment：支付记录（模拟）
- Review：评价（订单完成后一次性；评分统计）

核心对象关系类图（概要）：

### 综合生活助手平台 — 核心领域类图（概要）



### 3.3.2 物理结构设计要点

物理存储采用“关系型数据库 + 缓存”组合：

- MySQL：持久化存储用户、商家、菜品、订单、支付、评价等核心数据
  - 金额字段统一使用“分”（BIGINT/INT）存储，避免浮点精度问题
  - 关键唯一约束：user.phone、order.order\_no、payment.order\_id、review.order\_id
  - 时间字段：created\_at、updated\_at；软删除字段：is\_deleted
- Redis：提升热点访问与保存短期数据
  - 购物车：Key 建议 cart:{userId}:{merchantId}，TTL=7天
  - JWT黑名单：Key 建议 jwt:blacklist:{jti} 或 jwt:blacklist:{tokenHash}，TTL=Token剩余有效期
  - 热点商家/菜品缓存：Key 建议 merchant:hot:{id}、product:list:{merchantId}，TTL=30分钟

访问方式：后端通过 ORM（MyBatis-Plus）访问 MySQL；通过 Redis 客户端访问缓存。安全方面：数据库/Redis 凭据与 JWT 密钥通过环境变量/独立配置管理，禁止硬编码。

### 3.3.3 数据结构与程序关系

数据结构与模块职责映射如下：

- 用户模块：User、UserAddress；Token黑名单（Redis）
- 商家模块：Merchant、Category、Product；热点缓存（Redis）
- 订单模块：Cart（Redis）、Order、OrderItem；订单状态机与超时取消任务
- 支付模块：Payment；订单状态更新（事务）
- 评价模块：Review；提交评价与商家评分聚合更新（事务）

所有写操作（下单、支付、评价、商家管理）均经过应用层 Service 进行业务校验与权限控制，再写入数据库/缓存。

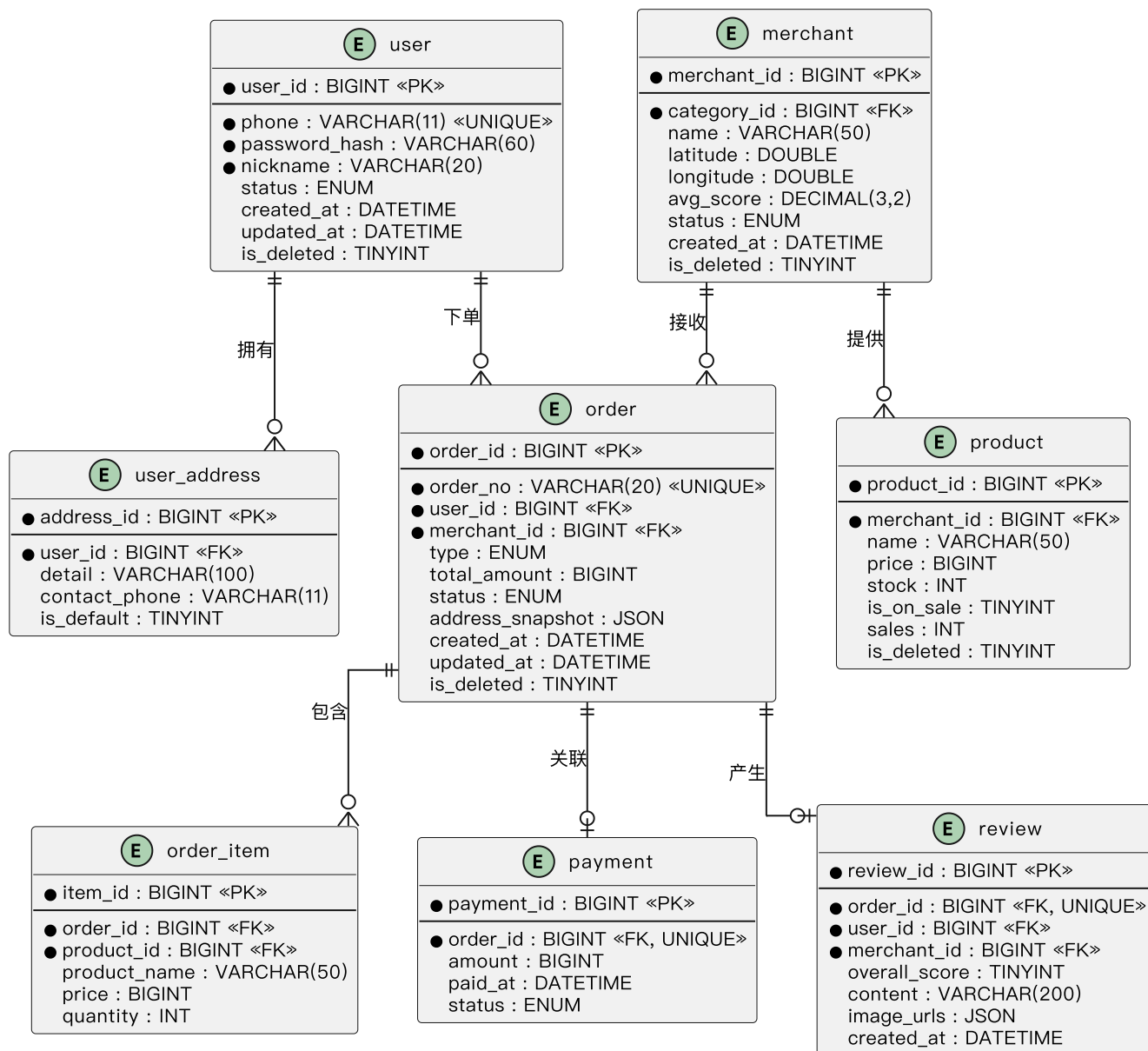
## 4 系统数据库设计

---

系统数据库采用 MySQL 8.x（字符集 utf8mb4），核心表包括：user、user\_address、category、merchant、product、order、order\_item、payment、review。

核心实体关系（ER图，概要）：

## 综合生活助手平台 — 核心ER图（概要）



数据库设计要点（概要）：

- 索引：user(phone)唯一索引；order(order\_no)唯一索引；payment(order\_id)唯一索引；review(order\_id)唯一索引
- 外键：order.user\_id、order.merchant\_id；order\_item.order\_id/product\_id；review.user\_id/merchant\_id；address.user\_id
- 事务：创建订单（order+item）、支付（payment+order状态更新）、提交评价（review+merchant.avg\_score更新）须在事务内完成

## 5 运行设计

### 5.1 运行模块组合

运行时模块组合建议如下：

- 前端模块：构建为静态资源（HTML/JS/CSS），由 Nginx 托管
- 后端模块：单体 Spring Boot 应用（JAR），按业务包划分模块（用户/商家/订单/支付/评价/商家后台）
- 数据模块：MySQL（持久化）+ Redis（缓存/购物车/黑名单）

在维护/降级状态下，系统可选择对写操作（下单/支付/评价/管理）进行统一拦截，仅保留浏览与查询能力，并向前端返回维护提示。

## 5.2运行控制

启动与运行控制要点：

- 启动顺序：MySQL/Redis →（可选）Nginx → Spring Boot 后端
- 配置管理：数据库连接、Redis地址/密码、JWT密钥等以环境变量或独立配置注入，禁止写入代码仓库
- 定时任务：
  - 支付超时处理：订单创建后30分钟未支付，自动取消（仅限待支付）
- 鉴权控制：
  - 登录成功签发JWT（有效期24小时）；登出将Token加入黑名单（Redis）
  - 连续5次登录失败锁定30分钟
- 统一异常处理：
  - 通过全局异常处理器统一返回 `{code, message, data}`，常用HTTP语义：400/401/403/404/409/500
  - Redis不可用时降级为不使用缓存（直查数据库），并记录告警日志
- 日志控制：
  - 结构化日志（含请求ID/时间戳/级别/类名/消息），按天滚动保留30天
  - 日志脱敏：手机号保留前3后4，其余使用“\*”替代；评价展示昵称脱敏

## 5.3运行时间

关键时间参数（概要）：

- JWT Token 有效期：24小时
- 登录失败锁定时长：30分钟
- 购物车（Redis）TTL：7天
- 热点缓存TTL：30分钟
- 外卖订单支付超时：30分钟（超时自动取消）

性能目标（测试环境）：查询接口 $\leq 1$ 秒、搜索 $\leq 2$ 秒；50并发下CPU $\leq 70\%$ 、内存 $\leq 80\%$ 。